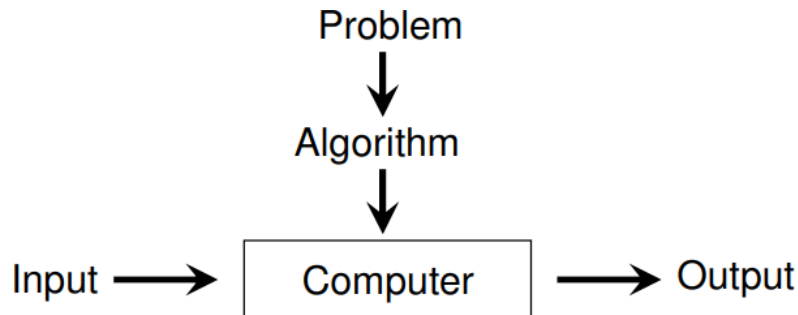


UNIT II

ALGORITHMIC DESIGN THINKING

2.1 ANALYSIS AND VERIFICATION

An algorithm is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time.



About Algorithms

- The non-ambiguity requirement for each step of an algorithm cannot be compromised
- The range of inputs for which an algorithm works has to be specified carefully.
- The same algorithm can be represented in several different ways
- There may exist several algorithms for solving the same problem.
- Can be based on very different ideas and can solve the problem with dramatically different speeds

Why Analyse an Algorithm?

The most straightforward reason for analyzing an algorithm is to discover its characteristics in order to evaluate its suitability for various applications or compare it with other algorithms for the same application. Moreover, the analysis of an algorithm can help us understand it better, and can suggest informed improvements. Algorithms tend to become shorter, simpler, and more elegant during the analysis process.

Computational Complexity:

The branch of theoretical computer science where the goal is to classify algorithms according to their efficiency and computational problems according to their inherent difficulty is known as computational complexity. Paradoxically, such classifications are typically not useful for predicting performance or for comparing algorithms in practical applications because they focus on order-of-growth worst-case performance. In this book, we focus on analyses that can be used to predict performance and compare algorithms.

Analysis of Algorithms:

A complete analysis of the running time of an algorithm involves the following steps:

- Implement the algorithm completely.
- Determine the time required for each basic operation.
- Identify unknown quantities that can be used to describe the frequency of execution of the basic operations.
- Develop a realistic model for the input to the program.
- Analyze the unknown quantities, assuming the modelled input.
- Calculate the total running time by multiplying the time by the frequency for each operation, then adding all the products.

Classical algorithm analysis on early computers could result in exact predictions of running times. Modern systems and algorithms are much more complex, but modern analyses are informed by the idea that exact analysis of this sort could be performed in principle.

Basic analysis operations in the context of algorithms involve understanding and evaluating the performance and behavior of algorithms. These operations are essential for assessing factors such as time complexity, space complexity, and correctness. Here are some key basic analysis operations:

Time Complexity Analysis:

Time complexity analysis determines the efficiency of an algorithm in terms of the time it takes to execute as a function of the size of the input. It involves analyzing the number of basic operations (such as comparisons, assignments, arithmetic operations) performed by the algorithm as the input size increases. Common notations used to represent time complexity include O (Big O), Ω (Omega), and Θ (Theta).

Space Complexity Analysis:

Space complexity analysis evaluates the amount of memory space required by an algorithm as a function of the input size. It involves analyzing the usage of memory resources, including variables, data structures, and auxiliary space, by the algorithm. Similar to time complexity, space complexity is often expressed using Big O notation.

Correctness Analysis:

Correctness analysis ensures that an algorithm produces the correct output for all possible inputs. It involves providing mathematical proofs or logical arguments to demonstrate the correctness of the algorithm under different scenarios. Techniques such as loop invariants, mathematical induction, and proof by contradiction are commonly used for correctness analysis.

Worst-case, Best-case, and Average-case Analysis:

Algorithms may exhibit different behaviors depending on the characteristics of the input data. Worst-case analysis evaluates the maximum time or space required by an algorithm for any input of a given size. Best-case analysis assesses the minimum time or space required for an algorithm, typically occurring for a specific input. Average-case analysis considers the expected performance of an algorithm over all possible inputs, often based on probabilistic assumptions.

Asymptotic Analysis:

Asymptotic analysis focuses on the behavior of an algorithm as the input size approaches infinity. It aims to capture the growth rate of the algorithm's time or space requirements without being concerned with specific constants or lower-order terms. Asymptotic analysis helps in comparing the relative efficiency of algorithms and understanding their scalability.

Empirical Analysis:

Empirical analysis involves practical experimentation and measurement of an algorithm's performance on real-world or synthetic datasets. It includes benchmarking the algorithm's execution time, memory usage, and other metrics using actual implementations and input data. Empirical analysis complements theoretical analysis and provides insights into the algorithm's performance in real-world scenarios.

By conducting these basic analysis operations, researchers, developers, and analysts can gain a comprehensive understanding of algorithms, identify optimization opportunities, and make informed decisions regarding algorithm selection and usage.

1.1 Order of growth

In algorithm analysis, the order of growth, often denoted using Big O notation, is a fundamental concept used to describe the behavior of an algorithm's time or space complexity as the size of the input approaches infinity. It provides a succinct way to express the upper bound on the algorithm's growth rate, disregarding constant factors and lower-order terms. The order of growth helps in comparing the relative efficiency of algorithms and understanding their scalability.

Types of asymptotic notations

Asymptotic notation is a fundamental concept in computer science and mathematics that allows us to describe the behavior of algorithms and functions as their input size approaches infinity. Asymptotic notation provides a simplified way to analyze and compare the efficiency of algorithms, focusing on their growth rates without being concerned with constant factors and lower-order terms. In this article, we delve into the properties of asymptotic notation, namely Big O, Omega, and Theta notation. By understanding these properties, we can gain valuable insights into algorithm performance and make informed decisions when designing or analyzing algorithms.

- Best Case – Minimum time for the execution.
- Average Case – Average time for the execution.
- Worst Case – Maximum time for the execution.

Asymptotic notations are mathematical tools to express the time complexity of algorithms for asymptotic analysis.

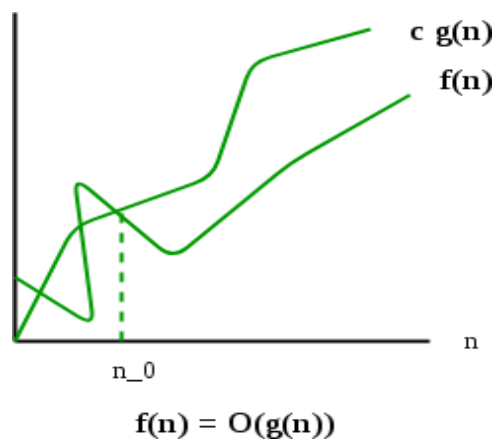
Three Main Asymptotic Notation

- O Big Oh Notation
- Ω Omega Notation
- θ Theta Notation

Big Oh Asymptotic Notation, O

Big-O notation represents the upper bound of the running time of an algorithm. Therefore, it gives the worst-case complexity of an algorithm.

- It is the most widely used notation for Asymptotic analysis.
- It specifies the upper bound of a function.
- The maximum time required by an algorithm or the worst-case time complexity.
- It returns the highest possible output value(big-O) for a given input.
- Big-O(Worst Case) It is defined as the condition that allows an algorithm to complete statement execution in the longest amount of time possible.



If $f(n)$ describes the running time of an algorithm, $f(n)$ is $O(g(n))$ if there exist a positive constant C and n_0 such that, $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0$

It returns the highest possible output value (big-O) for a given input. The execution time serves as an upper bound on the algorithm's time complexity.

Big O notation is used to describe the upper bound on the growth rate of an algorithm. It represents the worst-case scenario, indicating how the algorithm's time or space requirements grow as the size of the input increases. For example, if an algorithm has a time complexity of $O(n)$, it means that the algorithm's execution time grows linearly with the size of the input.

Common Orders of Growth:

- $O(1)$: Constant time complexity. The algorithm's execution time or space requirement remains constant regardless of the size of the input. Example: accessing an element in an array by index.
- $O(\log n)$: Logarithmic time complexity. The algorithm's performance grows logarithmically with the size of the input. Example: binary search.
- $O(n)$: Linear time complexity. The algorithm's performance grows linearly with the size of the input. Example: linear search.
- $O(n \log n)$: Log-linear time complexity. Commonly seen in efficient sorting algorithms such as mergesort and heapsort.
- $O(n^2)$: Quadratic time complexity. The algorithm's performance grows quadratically with the size of the input. Example: selection sort.
- $O(2^n)$: Exponential time complexity. The algorithm's performance grows exponentially with the size of the input. Example: brute-force search algorithms.
- $O(n!)$: Factorial time complexity. The algorithm's performance grows factorially with the size of the input. Example: traveling salesman problem solved by brute force.

Comparing Orders of Growth:

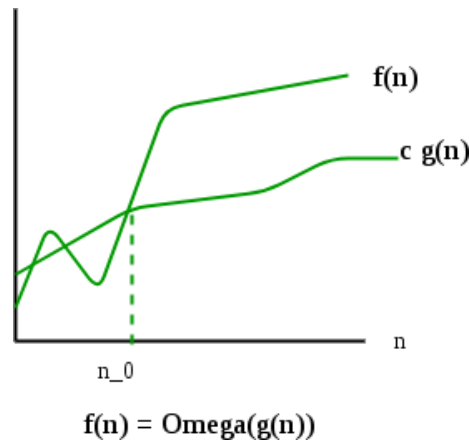
When comparing algorithms, the one with a lower order of growth is generally more efficient for large inputs. For example, an algorithm with $O(n \log n)$ time complexity is generally more efficient than an algorithm with $O(n^2)$ time complexity for sufficiently large inputs.

Asymptotic Analysis: The order of growth focuses on the behavior of an algorithm as the input size approaches infinity. It disregards constant factors and lower-order terms, providing a high-level understanding of an algorithm's scalability.

By analyzing the order of growth of algorithms, developers and analysts can make informed decisions about algorithm selection, optimization, and scalability, ensuring efficient problem-solving in various computational tasks.

Omega Notation Ω

Omega notation represents the lower bound of the running time of an algorithm. Thus, it provides the best case complexity of an algorithm.



The execution time serves as a lower bound on the algorithm's time complexity.

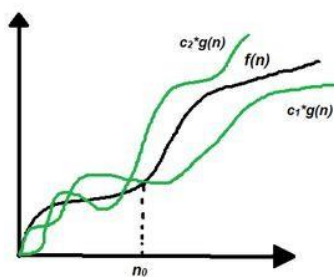
It is defined as the condition that allows an algorithm to complete statement execution in the shortest amount of time.

Let g and f be the function from the set of natural numbers to itself. The function f is said to be $\Omega(g)$, if there is a constant $c > 0$ and a natural number n_0 such that $c \cdot g(n) \leq f(n)$ for all $n \geq n_0$

Theta Notation Θ

Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average-case complexity of an algorithm. Theta (Average Case) You add the running times for each possible input combination and take the average in the average case.

Let g and f be the function from the set of natural numbers to itself. The function f is said to be $\Theta(g)$, if there are constants $c_1, c_2 > 0$ and a natural number n_0 such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$



Space and Time Complexity

The analysis is a process of estimating the efficiency of an algorithm. There are two fundamental parameters based on which we can analysis the algorithm:

- **Space Complexity:** The space complexity can be understood as the amount of space required by an algorithm to run to completion.
- **Time Complexity:** Time complexity is a function of input size n that refers to the amount of time needed by an algorithm to run to completion.

In general, if there is a problem **P1**, then it may have many solutions, such that each of these solutions is regarded as an algorithm. So, there may be many algorithms such as **A1, A2, A3, ..., An**.

Before you implement any algorithm as a program, it is better to find out which among these algorithms are good in terms of time and memory. It would be best to analyze every algorithm in terms of **Time** that relates to which one could execute faster and **Memory** corresponding to which one will take less memory.

So, the Design and Analysis of Algorithm talks about how to design various algorithms and how to analyze them. After designing and analyzing, choose the best algorithm that takes the least time and the least memory and then implement it as a program in C.

Generally, we make three types of analysis, which is as follows:

- **Worst-case time complexity:** For 'n' input size, the worst-case time complexity can be defined as the maximum amount of time needed by an algorithm to complete its execution. Thus, it is nothing but a function defined by the maximum number of steps performed on an instance having an input size of n.
- **Average case time complexity:** For 'n' input size, the average-case time complexity can be defined as the average amount of time needed by an algorithm to complete its execution. Thus, it is nothing but a function defined by the average number of steps performed on an instance having an input size of n.
- **Best case time complexity:** For 'n' input size, the best-case time complexity can be defined as the minimum amount of time needed by an algorithm to complete its execution. Thus, it is nothing but a function defined by the minimum number of steps performed on an instance having an input size of n.

Complexity of Algorithm

The term algorithm complexity measures how many steps are required by the algorithm to solve the given problem. It evaluates the order of count of operations executed by an algorithm as a function of input data size. To assess the complexity, the order (approximation) of the count of operation is always considered instead of counting the exact steps.

$O(f)$ notation represents the complexity of an algorithm, which is also termed as an Asymptotic notation or "**Big O**" notation. Here the f corresponds to the function whose size is the same as that of the input data. The complexity of the asymptotic computation $O(f)$ determines in which order the resources such as CPU time, memory, etc. are consumed by the algorithm that is articulated as a function of the size of the input data.

The complexity can be found in any form such as constant, logarithmic, linear, $n \cdot \log(n)$, quadratic, cubic, exponential, etc. It is nothing but the order of constant, logarithmic, linear and so on, the number of steps encountered for the completion of a particular algorithm. To make it even more precise, we often call the complexity of an algorithm as "running time".

Typical Complexities of an Algorithm

Constant Complexity:

It imposes a complexity of **$O(1)$** . It undergoes an execution of a constant number of steps like 1, 5, 10, etc. for solving a given problem. The count of operations is independent of the input data size.

Logarithmic Complexity:

It imposes a complexity of **$O(\log(N))$** . It undergoes the execution of the order of $\log(N)$ steps. To perform operations on N elements, it often takes the logarithmic base as 2. For $N = 1,000,000$, an algorithm that has a complexity of $O(\log(N))$ would undergo 20 steps (with a constant precision). Here, the logarithmic base does not hold a necessary consequence for the operation count order, so it is usually omitted.

Linear Complexity:

- It imposes a complexity of **$O(N)$** . It encompasses the same number of steps as that of the total number of elements to implement an operation on N elements. For example, if there exist 500 elements, then it will take about 500 steps. Basically, in linear complexity, the number of elements linearly depends on the number of steps. For example, the number of steps for N elements can be $N/2$ or $3*N$.
- **$O(N \log(N))$** : It also imposes a run time of **$O(n \log(n))$** . It undergoes the execution of the order $N \log(N)$ on N number of elements to solve the given problem. For a given 1000 elements, the linear complexity will execute 10,000 steps for solving a given problem.
- **Quadratic Complexity:** It imposes a complexity of **$O(n^2)$** . For N input data size, it undergoes the order of N^2 count of operations on N number of elements for solving a given problem.
- If $N = 100$, it will endure 10,000 steps. In other words, whenever the order of operation tends to have a quadratic relation with the input data size, it results in quadratic complexity. For example, for N number of elements, the steps are found to be in the order of $3*N^2/2$.
- **Cubic Complexity:** It imposes a complexity of **$O(n^3)$** . For N input data size, it executes the order of N^3 steps on N elements to solve a given problem. For example, if there exist 100 elements, it is going to execute 1,000,000 steps.
- **Exponential Complexity:** It imposes a complexity of **$O(2^n)$, $O(N!)$, $O(n^k)$,** For N elements, it will execute the order of count of operations that is exponentially dependable on the input data size.

For example, if $N = 10$, then the exponential function 2^N will result in 1024. Similarly, if $N = 20$, it will result in 1048 576, and if $N = 100$, it will result in a number having 30 digits.

The exponential function $N!$ grows even faster; for example, if $N = 5$ will result in 120. Likewise, if $N = 10$, it will result in 3,628,800 and so on.

Since the constants do not hold a significant effect on the order of count of operation, so it is better to ignore them. Thus, to consider an algorithm to be linear and equally efficient, it must undergo N , $N/2$ or $3*N$ count of operation, respectively, on the same number of elements to solve a particular problem.

Space and time complexity are fundamental concepts in algorithm analysis, which help in understanding how algorithms behave in terms of memory usage and execution time as the input size grows.

Time Complexity:

Time complexity refers to the amount of time an algorithm takes to complete as a function of the size of its input. It provides an estimation of the number of operations performed by the algorithm relative to the input size. Time complexity is typically expressed using Big O notation.

Example 1: Linear Search Algorithm

```
def linear_search(arr, target):
```

```
    for i in range(len(arr)):
```

```
        if arr[i] == target:
```

```
            return i
```

```
    return -1
```

Time Complexity: $O(n)$ - Linear Time

In this example, the time complexity of the linear search algorithm is $O(n)$, where 'n' represents the size of the input array. This is because the algorithm iterates through each element of the array once to find the target element.

Example 2: Binary Search Algorithm (on a sorted array)

```
def binary_search(arr, target):
```

```
    low = 0
```

```
    high = len(arr) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if arr[mid] == target:
```

```
            return mid
```

```
        elif arr[mid] < target:
```

```
            low = mid + 1
```

Time Complexity: $O(\log n)$ - Logarithmic Time

In this example, the time complexity of the binary search algorithm is $O(\log n)$, where 'n' represents the size of the input array. This is because the algorithm halves the search space in each iteration, resulting in a logarithmic growth rate.

Space Complexity:

Space complexity refers to the amount of memory space required by an algorithm to execute as a function of the size of its input. It includes both the space required by the algorithm itself (e.g., variables, data structures) and any additional space used during its execution (e.g., recursion stack, auxiliary space). Like time complexity, space complexity is also expressed using Big O notation.

Example 3: Linear Search Algorithm

```
def linear_search(arr, target):
    # No additional space used besides function arguments and variables
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

# Space Complexity:  $O(1)$  - Constant Space
```

In this example, the space complexity of the linear search algorithm is $O(1)$, indicating that the amount of space used by the algorithm remains constant regardless of the size of the input array. This is because the algorithm does not require any additional memory space proportional to the input size.

Example 4: Recursive Factorial Calculation

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)

# Space Complexity:  $O(n)$  - Linear Space (due to recursion stack)
```

In this example, the space complexity of the factorial calculation algorithm is $O(n)$, where 'n' represents the input value. This is because the algorithm uses recursive function calls, which consume memory space on the call stack proportional to the depth of recursion (i.e., the input value 'n').

Understanding time and space complexity is crucial for evaluating the efficiency and scalability of algorithms, enabling developers to make informed decisions when selecting or designing algorithms for various applications.

2.2 BRUTE FORCE

A brute force algorithm is a simple, comprehensive search strategy that systematically explores every option until a problem's answer is discovered. It's a generic approach to problem-solving that's employed when the issue is small enough to make an in-depth investigation possible. However, because of their high temporal complexity, brute force techniques are inefficient for large-scale issues.

Methodical Listing: Brute force algorithms investigate every potential solution to an issue, usually in an organized and detailed way. This involves attempting each option in a specified order.

Relevance: When the issue space is small and easily explorable in a fair length of time, brute force is the most appropriate method. The temporal complexity of the algorithm becomes unfeasible for larger issue situations.

Not using optimization or heuristics: Brute force algorithms don't use optimization or heuristic approaches. They depend on testing every potential outcome without ruling out any using clever pruning or heuristics.

Features of the brute force algorithm

- It is an intuitive, direct, and straightforward technique of problem-solving in which
- all the possible ways or all the possible solutions to a given problem are enumerated.
- Many problems are solved in day-to-day life using the brute force strategy, for example, exploring all the paths to a nearby market to find the minimum shortest path.
- Arranging the books in a rack using all the possibilities to optimize the rack spaces, etc.
- Daily life activities use a brute force nature, even though optimal algorithms are also possible.

Pros and Cons of brute force algorithm:

Pros:

- The brute force approach is a guaranteed way to find the correct solution by listing all the possible candidate solutions for the problem.
- It is a generic method and not limited to any specific domain of problems.
- The brute force method is ideal for solving small and simpler problems.
- It is known for its simplicity and can serve as a comparison benchmark.

Cons:

- The brute force approach is inefficient. For real-time problems, algorithm analysis often goes above the $O(N!)$ order of growth.

- This method relies more on compromising the power of a computer system for solving a problem than on a good algorithm design.
- Brute force algorithms are slow.
- Brute force algorithms are not constructive or creative compared to algorithms that are constructed using some other design paradigms.

Applications of Brute Force Algorithm:

String Matching:

The problem of matching patterns in strings is central to database and text processing applications. The problem will be specified as: given an input text string t of length n , and a pattern string p of length m , find the first (or all) instances of the pattern in the text.

The simplest algorithm for string matching is a brute force algorithm, where we simply try to match the first character of the pattern with the first character of the text, and if we succeed, try to match the second character, and so on; if we hit a failure point, slide the pattern over one character and try again. When we find a match, return its starting location. Java code for the brute force method:

```
for (int i = 0; i < n-m; i++) {
    int j = 0;
    while (j < m && t[i+j] == p[j]) {
        j++;
    }
    if (j == m) return i; }
System.out.println("No match found");
return -1;
```

The outer loop is executed at most $n-m+1$ times, and the inner loop m times, for each iteration of the outer loop. Therefore, the running time of this algorithm is in $O(nm)$.

Travelling Salesman Problem:

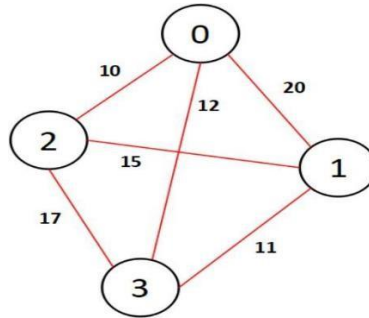
Travelling Salesman Problem is based on a real life scenario, where a salesman from a company has to start from his own city and visit all the assigned cities exactly once and return to his home till the end of the day. The exact problem statement goes like this, "Given a set of cities and distance between every pair of cities, the problem is to find the shortest possible route that visits every city exactly once and returns to the starting point."

There are two important things to be cleared about in this problem statement,

- Visit every city exactly once
- Cover the shortest path

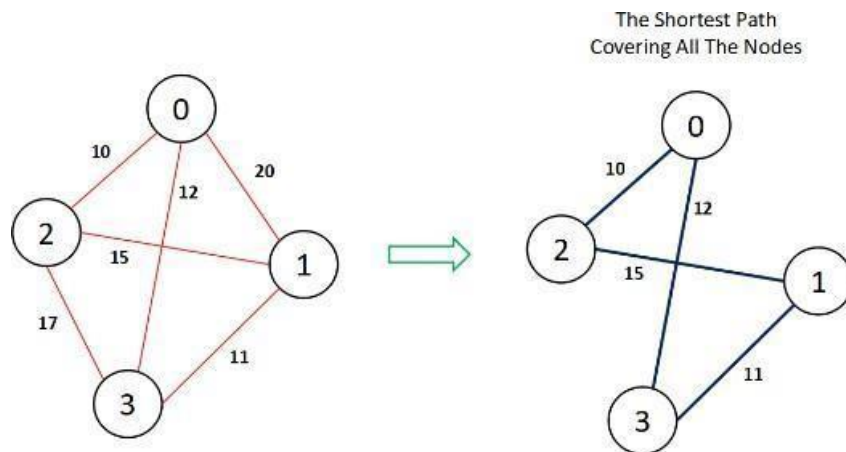
Visualizing the problem

We can visualize the problem by creating a graph data structure having some nodes and weighted edges as path lengths. For example, have a look at the following image,



For example - Node 2 to Node 3 takes a weighted edge of 17.

We need to find the shortest path covering all the nodes exactly once, which is highlighted in the figure below for the above graph.



Steps to Solve the Problem:

There are few classical and easy steps that we must follow to solve the TSP problem,

- Finding Adjacent matrix of the graph, which will act as an input.
- Performing the shortest_path algorithm, by coding out a function.
- Understanding next_permutation.

Step-1 - Finding Adjacent Matrix of the Graph

You will need a two dimensional array for getting the Adjacent Matrix of the given graph. Here are the steps;

- Get the total number of nodes and total number of edges in two variables namely num_nodes and num_edges.
- Create a multidimensional array edges_list having the dimension equal to num_nodes * num_nodes

- Run a loop `num_nodes` time and take two inputs namely `first_node` and `second_node` * everytime as two nodes having an edge between them and place the `edges_list[first_node][second_node]` position equal to 1.
- Finally after the loop executes we have an adjacent matrix available i.e `edges_list`.

Step - 2 - Performing The Shortest Path Algorithm

The most important step in designing the core algorithm is this one, let's have a look at the pseudocode of the algorithm below.

- Considering a starting source city, from where the salesman will start. We can consider any city as the starting point and by default we have considered 0 as the starting point here.
- Generating the permutation of the rest cities. Suppose we have total N nodes and we have considered one node as the source, then we need to generate the rest $(N-1)!$ (Factorial of $N-1$) permutations.
- We need to calculate the edge sum (path sum) for every permutation and take a track of the minimum path sum with each permutation.
- Return the minimum edge cost.

Step - 3 - Understanding next_permutation

It's a good practise to understand the functions from Standard Template Library on what they take as argument, their working mechanism and their output. In this algorithm we have used a function named `next_permutation()`, which takes two Bidirectional Iterators namely, (here `vector::iterator`) `nodes.begin()` and `nodes.end()`.

This functions returns a Boolean Type (i.e. either true or false).

Working Mechanism:

This function rearranges the objects in `[nodes.begin(),nodes.end())`, where the `[]` represents both iterator inclusive, in a lexicographical order. If there exists a greater lexicographical arrangement than the current arrangement, then the function returns true else it returns false.

Lexicographical order is also known as dictionary order in mathematics.

Complexity:

The time complexity of the algorithm is dependent upon the number of nodes. If the number of nodes is n then the time complexity will be proportional to $n!$ (factorial of n) i.e. $O(n!)$.

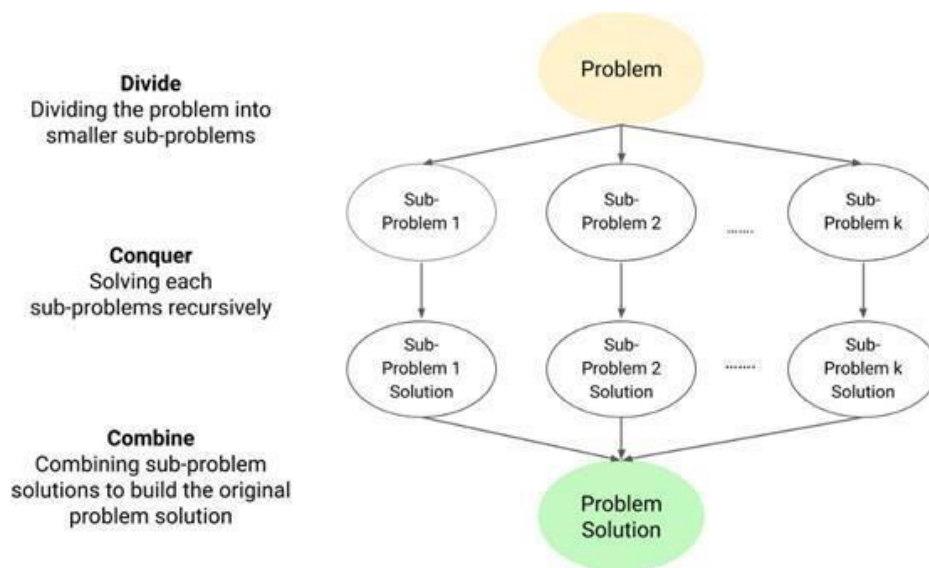
The most amount of space in this graph algorithm is taken by the adjacent matrix which is a $n * n$ two dimensional matrix, where n is the number of nodes. Hence the space complexity is $O(n^2)$.

2.3 DIVIDE AND CONQUER

Divide and Conquer is an algorithmic pattern. In algorithmic methods, the design is to take a dispute on a huge input, break the input into minor pieces, decide the problem on each of the small pieces, and then merge the piecewise solutions into a global solution. This mechanism of solving the problem is called the Divide & Conquer Strategy.

Divide and Conquer algorithm consists of a dispute using the following three steps.

1. **Divide** the original problem into a set of sub problems.
2. **Conquer**: Solve every sub problem individually, recursively.
3. **Combine**: Put together the solutions of the sub problems to get the solution to the whole problem.



Applications of Divide and Conquer Approach:

Following algorithms are based on the concept of the Divide and Conquer Technique:

1. **Binary Search:** The binary search algorithm is a searching algorithm, which is also called a half-interval search or logarithmic search. It works by comparing the target value with the middle element existing in a sorted array. After making the comparison, if the value differs, then the half that cannot contain the target will eventually eliminate, followed by continuing the search on the other half. We will again consider the middle element and compare it with the target value. The process keeps on repeating until the target value is met. If we found the other half to be empty after ending the search, then it can be concluded that the target is not present in the array.
2. **Quicksort:** It is the most efficient sorting algorithm, which is also known as partition-exchange sort. It starts by selecting a pivot value from an array followed by dividing the rest of the array elements into two sub-arrays. The partition is made by comparing each of the elements with the pivot value. It compares whether the element holds a greater value or lesser value than the pivot and then sort the arrays recursively.

3. **Merge Sort:** It is a sorting algorithm that sorts an array by making comparisons. It starts by dividing an array into sub-array and then recursively sorts each of them. After the sorting is done, it merges them back.
4. **Closest Pair of Points:** It is a problem of computational geometry. This algorithm emphasizes finding out the closest pair of points in a metric space, given n points, such that the distance between the pair of points should be minimal.
5. **Strassen's Algorithm:** It is an algorithm for matrix multiplication, which is named after Volker Strassen. It has proven to be much faster than the traditional algorithm when works on large matrices.
6. **Cooley-Tukey Fast Fourier Transform (FFT) algorithm:** The Fast Fourier Transform algorithm is named after J. W. Cooley and John Turkey. It follows the Divide and Conquer Approach and imposes a complexity of $O(n \log n)$.
7. **Karatsuba algorithm for fast multiplication:** It is one of the fastest multiplication algorithms of the traditional time, invented by Anatoly Karatsuba in late 1960 and got published in 1962. It multiplies two n -digit numbers in such a way by reducing it to at most single-digit.

Advantages of Divide and Conquer

- Divide and Conquer tend to successfully solve one of the biggest problems, such as the
- Tower of Hanoi, a mathematical puzzle. It is challenging to solve complicated problems for which you have no basic idea, but with the help of the divide and conquer approach,
- It has lessened the effort as it works on dividing the main problem into two halves and then solve them recursively. This algorithm is much faster than other algorithms.
- It efficiently uses cache memory without occupying much space because it solves simple sub problems within the cache memory instead of accessing the slower main memory.
- It is more proficient than that of its counterpart Brute Force technique.
- Since these algorithms inhibit parallelism, it does not involve any modification and is handled by systems incorporating parallel processing.

Disadvantages of Divide and Conquer

- Since most of its algorithms are designed by incorporating recursion, so it necessitates high memory management.
- An explicit stack may overuse the space.
- It may even crash the system if the recursion is performed rigorously greater than the stack present in the CPU.

Merge Sort

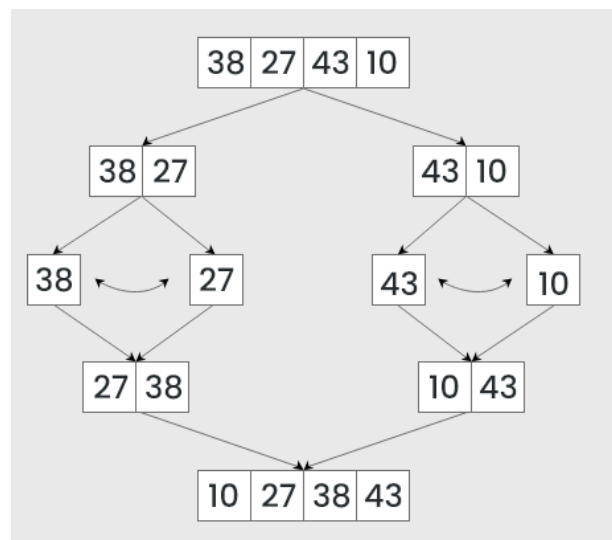
Merge sort is a sorting algorithm that falls under the category of Divide and Conquer technique. It is one of the best sorting techniques that successfully build a recursive algorithm.

1. Step1: The merge sort algorithm iteratively divides an array into equal halves until we achieve an atomic value. In case if there are an odd number of elements in an array, then one of the halves will have more elements than the other half.
2. Step2: After dividing an array into two subarrays, we will notice that it did not hamper the order of elements as they were in the original array. After now, we will further divide these two arrays into other halves.
3. Step3: Again, we will divide these arrays until we achieve an atomic value, i.e., a value that cannot be further divided.
4. Step4: Next, we will merge them back in the same way as they were broken down.
5. Step5: For each list, we will first compare the element and then combine them to form a new sorted list.
6. Step6: In the next iteration, we will compare the lists of two data values and merge them back into a list of found data values, all placed in a sorted manner.

Merge sort is a sorting algorithm that follows the divide-and-conquer approach. It works by recursively dividing the input array into smaller subarrays and sorting those subarrays then merging them back together to obtain the sorted array.

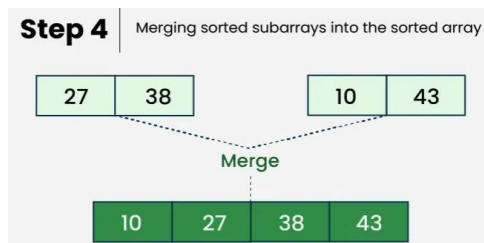
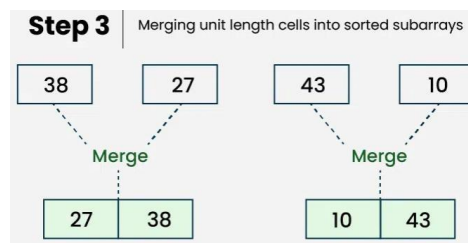
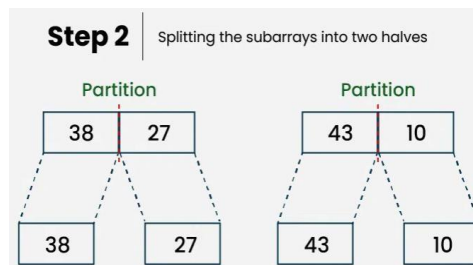
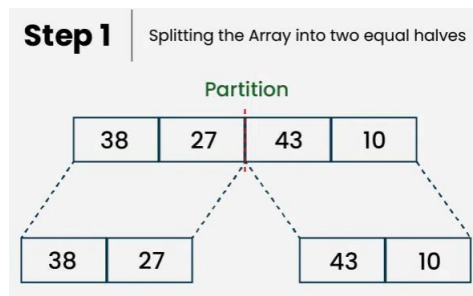
In simple terms, we can say that the process of merge sort is to divide the array into two halves, sort each half, and then merge the sorted halves back together. This process is repeated until the entire array is sorted.

Example:



Here's a step-by-step explanation of how merge sort works:

- **Divide:** Divide the list or array recursively into two halves until it can no more be divided.
- **Conquer:** Each subarray is sorted individually using the merge sort algorithm.
- **Merge:** The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.



Let's look at the working of above example:

Divide:

[38, 27, 43, 10] is divided into [38, 27] and [43, 10] .

[38, 27] is divided into [38] and [27] .

[43, 10] is divided into [43] and [10] .

Conquer:

[38] is already sorted.

[27] is already sorted.

[43] is already sorted.

[10] is already sorted.

Merge:

Merge [38] and [27] to get [27, 38] .

Merge [43] and [10] to get [10,43] .

Merge [27, 38] and [10,43] to get the final sorted list [10, 27, 38, 43]

Therefore, the sorted list is [10, 27, 38, 43].

Efficiency:

The divide and conquer strategy is a widely used algorithmic paradigm that solves a problem by breaking it into smaller subproblems, solving each subproblem recursively, and then combining the results. Its efficiency can be evaluated in terms of time complexity and space complexity:

1. Time Complexity

- The time complexity of a divide and conquer algorithm depends on:
- The number of subproblems into which the problem is divided.
- The size of each subproblem.
- The cost of dividing the problem and combining the results.

Examples:

- **Merge Sort:** $T(n)=2T(n/2)+O(n)$ $T(n) = 2T(n/2) + O(n)$ $T(n)=2T(n/2)+O(n) \rightarrow$ Time Complexity: $O(n \log n)$ $O(n \log n)$ $O(n \log n)$.
- **Binary Search:** $T(n)=T(n/2)+O(1)$ $T(n) = T(n/2) + O(1)$ $T(n)=T(n/2)+O(1) \rightarrow$ Time Complexity: $O(\log n)$ $O(\log n)$ $O(\log n)$.

2. Space Complexity

Space complexity in divide and conquer depends on:

1. **Auxiliary Space:** Additional memory used during recursion.
2. **Recursive Stack Space:** Memory required to maintain the recursive function calls.

Analysis:

- Each recursive call consumes stack space proportional to the depth of the recursion.
- For most divide and conquer algorithms, the depth of recursion is $O(\log n)$ ($O(\log n)$ when the problem size reduces geometrically).

Examples:

- **Merge Sort:** Requires $O(n)$ auxiliary space for merging and $O(\log n)$ stack space $\rightarrow$ Total Space: $O(n + \log n)$.
- **Quick Sort:** Requires $O(\log n)$ stack space in the best case and $O(n)$ in the worst case $\rightarrow$ Total Space: $O(\log n)$ (best case).

2.4 GREEDY ALGORITHM

A Greedy algorithm is an approach to solving a problem that selects the most appropriate option based on the current situation. This algorithm ignores the fact that the current best result may not bring about the overall optimal result. Even if the initial decision was incorrect, the algorithm never reverses it.

This simple, intuitive algorithm can be applied to solve any optimization problem which requires the maximum or minimum optimum result. The best thing about this algorithm is that it is easy to understand and implement. The runtime complexity associated with a greedy solution is pretty reasonable. However, you can implement a greedy solution only if the problem statement follows two properties mentioned below:

- **Greedy Choice Property:** Choosing the best option at each phase can lead to a global (overall) optimal solution.
- **Optimal Substructure:** If an optimal solution to the complete problem contains the optimal solutions to the subproblems, the problem has an optimal substructure.

Moving forward, we will learn how to create a greedy solution for a problem that adheres to the principles listed above.

Steps for Creating a Greedy Algorithm

By following the steps given below, you will be able to formulate a greedy solution for the given problem statement:

- **Step 1:** In a given problem, find the best substructure or subproblem.
- **Step 2:** Determine what the solution will include (e.g., largest sum, shortest path).
- **Step 3:** Create an iterative process for going over all subproblems and creating an optimum solution.

Let's take up a real-world problem and formulate a greedy solution for it.

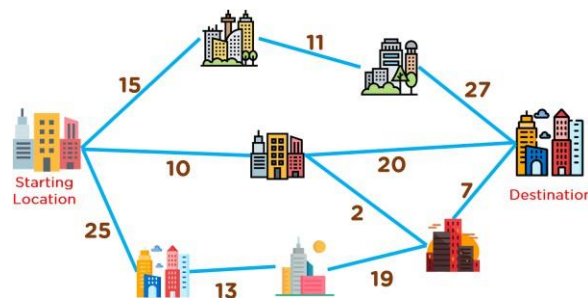
Problem:

Alex is a very busy person. He has set aside time T to accomplish some interesting tasks. He wants to do as many tasks as possible in this allotted time T . For that, he has created an array A of timestamps to complete a list of items on his itinerary. Now, here we need to figure out how many things Alex can complete in the T time he has. Approach to Build a Solution: This given problem is a straightforward greedy problem. In each iteration, we will have to pick the items from array A that will take the least amount of time to accomplish a task while keeping two variables in mind: `current_Time` and `number_Of_Things`. To generate a solution, we will have to carry out the following steps.

- Sort the array A in ascending order.
- Select one timestamp at a time.
- After picking up the timestamp, add the timestamp value to `current_Time`.
- Increase `number_Of_Things` by one.
- Repeat steps 2 to 4 until the `current_Time` value reaches T .

Example of Greedy Algorithm

Problem Statement: Find the best route to reach the destination city from the given starting point using a greedy method.

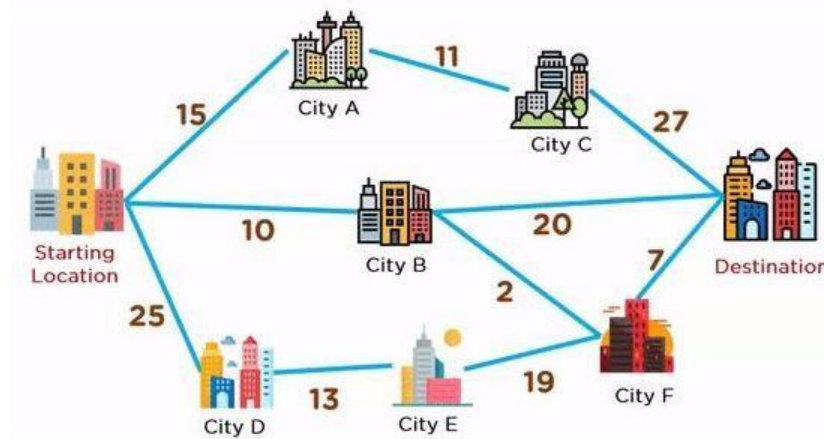


Greedy Solution: In order to tackle this problem, we need to maintain a graph structure. And for that graph structure, we'll have to create a tree structure, which will serve as the answer to this problem.

The steps to generate this solution are given below:

- Start from the source vertex.
- Pick one vertex at a time with a minimum edge weight (distance) from the source vertex.
- Add the selected vertex to a tree structure if the connecting edge does not form a cycle.
- Keep adding adjacent fringe vertices to the tree until you reach the destination vertex.

The animation given below explains how paths will be picked up in order to reach the destination city.



Limitations of Greedy Algorithm

Factors listed below are the limitations of a greedy algorithm:

- The greedy algorithm makes judgments based on the information at each iteration without considering the broader problem; hence it does not produce the best answer for every problem.
- The problematic part for a greedy algorithm is analyzing its accuracy. Even with the proper solution, it is difficult to demonstrate why it is accurate.
- Optimization problems (Dijkstra's Algorithm) with negative graph edges cannot be solved using a greedy algorithm.

Moving forward, let's look at some applications of a greedy algorithm.

Applications of Greedy Algorithm

Following are few applications of the greedy algorithm:

- Used for Constructing Minimum Spanning Trees: Prim's and Kruskal's Algorithms used to construct minimum spanning trees are greedy algorithms.
- Used to Implement Huffman Encoding: A greedy algorithm is utilized to build a Huffman tree that compresses a given image, spreadsheet, or video into a lossless compressed file.
- Used to Solve Optimization Problems: Graph - Map Coloring, Graph - Vertex Cover, Knapsack Problem, Job Scheduling Problem, and activity selection problem are classic optimization problems solved using a greedy algorithmic paradigm.

Characteristics of a Greedy Method

The greedy method is a simple and straightforward way to solve optimization problems. It involves making the locally optimal choice at each stage with the hope of finding the global optimum. The main advantage of the greedy method is that it is easy to implement and understand. However, it is not always guaranteed to find the best solution and can be quite slow.

The greedy method works by making the locally optimal choice at each stage in the hope of finding the global optimum. This can be done by either minimizing or maximizing the objective function at each step. The main advantage of the greedy method is that it is relatively easy to implement and understand. However, there are some disadvantages to using this method. First, the greedy method is not guaranteed to find the best solution. Second, it can be quite slow. Finally, it is often difficult to prove that the greedy method will indeed find the global optimum.

One of the most famous examples of the greedy method is the knapsack problem. In this problem, we are given a set of items, each with a weight and a value. We want to find the subset of items that maximizes the value while minimizing the weight. The greedy method would simply take the item with the highest value at each step. However, this might not be the best solution. For example, consider the following set of items:

Item 1: Weight = 2, Value = 6

Item 2: Weight = 2, Value = 3

Item 3: Weight = 4, Value = 5

The greedy method would take Item 1 and Item 3, for a total value of 11. However, the optimal solution would be to take Item 2 and Item 3, for a total value of 8. Thus, the greedy method does not always find the best solution.

There are many other examples of the greedy method. The most famous one is probably the Huffman coding algorithm, which is used to compress data. In this algorithm, we are given a set of symbols, each with a weight. We want to find the subset of symbols that minimizes the average length of the code. The greedy method would simply take the symbol with the lowest weight at each step. However, this might not be the best solution. For example, consider the following set of symbols:

Symbol 1: Weight = 2, Code = 00

Symbol 2: Weight = 3, Code = 010

Symbol 3: Weight = 4, Code = 011

The greedy method would take Symbol 1 and Symbol 3, for a total weight of 6. However, the optimal solution would be to take Symbol 2 and Symbol 3, for a total weight of 7. Thus, the greedy method does not always find the best solution.

The greedy method is a simple and straightforward way to solve optimization problems. However, it is not always guaranteed to find the best solution and can be quite slow. When using the greedy method, it is important to keep these disadvantages in mind.

Components of a Greedy Algorithm

There are four key components to any greedy algorithm:

1. A set of candidate solutions (typically represented as a graph)

2. A way of ranking the candidates according to some criteria
3. A selection function that picks the best candidate from the set, according to the ranking
4. A way of "pruning" the set of candidates, so that it doesn't contain any solutions that are worse than the one already chosen.

The first two components are straightforward - the candidate solutions can be anything, and the ranking criteria can be anything as well. The selection function is usually just a matter of picking the candidate with the highest ranking.

The pruning step is important, because it ensures that the algorithm doesn't waste time considering candidates that are already known to be worse than the best one found so far. Without this step, the algorithm would essentially be doing a brute-force search of the entire solution space, which would be very inefficient.

Pseudo Code of Greedy Algorithms

One example of pseudo code for a greedy algorithm is given below:

```
function GreedyAlgorithm(problem)
{
// currentState = initial state of problem
while (currentState != goalState)
{
nextState =chooseNextState(currentState);
currentState = nextState;
}
return currentState;
}
```

The above pseudo code shows the basic structure of a greedy algorithm. The first step is to set the current state to the initial state of the problem. Next, we keep looping until the current state is equal to the goal state. Inside the loop, we choose the next state that we want to move to. This is done by using a function called chooseNextState(). Finally, we set the current state to be equal to the next state that we have just chosen and return to the goal state.

The pseudo code for the chooseNextState() function is given below:

```
function chooseNextState(currentState) { // find all possible next states nextStates =
findAllPossibleNextStates(currentState); // choose the next state that is closest to the goal state
bestState = null; for (state in nextStates) { if (isCloserToGoal(state, bestState)) { bestState =
state;
} } return bestState; }
```


The pseudo code for the `chooseNextState()` function shows how we can choose the next state that is closest to the goal state. First, we find all of the possible next states that we could move to from the current state. Next, we loop through all of the possible next states and compare each one to see if it is closer to the goal state than the best state that we have found so far. If it is, then we set the best state to be equal to the new state. Finally, we return the best state that we have found.

The above pseudo code shows how a greedy algorithm works in general. However, it is important to note that not all problems can be solved using a greedy algorithm. In some cases, it may be necessary to use a different type of algorithm altogether.

Disadvantages of Using Greedy Algorithms

The main disadvantage of using a greedy algorithm is that it may not find the optimal solution to a problem. In other words, it may not produce the best possible outcome. Additionally, greedy algorithms can be very sensitive to changes in input data — even a small change can cause the algorithm to produce a completely different result. Finally, greedy algorithms can be difficult to implement and understand.

Fractional Knapsack Problem

Given the weights and profits of N items, in the form of $\{\text{profit}, \text{weight}\}$ put these items in a knapsack of capacity W to get the maximum total profit in the knapsack. In Fractional Knapsack, we can break items for maximizing the total value of the knapsack.

The basic idea of the greedy approach is to calculate the ratio profit/weight for each item and sort the item on the basis of this ratio. Then take the item with the highest ratio and add them as much as we can (can be the whole element or a fraction of it).

This will always give the maximum profit because, in each step it adds an element such that this is the maximum possible profit for that much weight.

Consider the example: $\text{arr}[] = \{\{100, 20\}, \{60, 10\}, \{120, 30\}\}$, $W = 50$.

Sorting: Initially sort the array based on the profit/weight ratio. The sorted array will be $\{\{60, 10\}, \{100, 20\}, \{120, 30\}\}$.

Iteration:

For $i = 0$, weight = 10 which is less than W . So add this element in the knapsack. profit = 60 and remaining $W = 50 - 10 = 40$.

For $i = 1$, weight = 20 which is less than W . So add this element too. profit = $60 + 100 = 160$ and remaining $W = 40 - 20 = 20$.

For $i = 2$, weight = 30 is greater than W . So add $20/30$ fraction = $2/3$ fraction of the element.

Therefore profit = $2/3 * 120 + 160 = 80 + 160 = 240$ and remaining W becomes 0. So the final profit becomes 240 for $W = 50$.

Follow the given steps to solve the problem using the above approach:

- Calculate the ratio (profit/weight) for each item.
- Sort all the items in decreasing order of the ratio.
- Initialize $res = 0$, $curr_cap = given_cap$.
- Do the following for every item i in the sorted order:
- If the weight of the current item is less than or equal to the remaining capacity then add the value of that item into the result
- Else add the current item as much as we can and break out of the loop.
- Return res .

Time Complexity: $O(N * \log N)$

Auxiliary Space: $O(N)$

2.5 BACKTRACKING ALGORITHM

- A backtracking algorithm is a problem-solving algorithm that uses a **brute force approach** for finding the desired output.
- The Brute force approach tries out all the possible solutions and chooses the desired/best solutions.
- The term backtracking suggests that if the current solution is not suitable, then backtrack and try other solutions. Thus, recursion is used in this approach

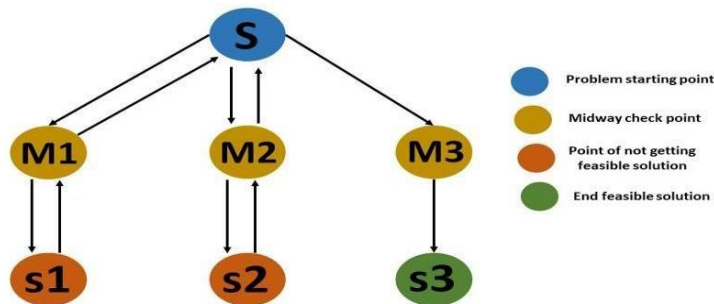
When to use a Backtracking algorithm?

When we have multiple choices, then we make the decisions from the available choices. In the following cases, we need to use the backtracking algorithm:

- A piece of sufficient information is not available to make the best choice, so we use the backtracking strategy to try out all the possible solutions.
- Each decision leads to a new set of choices. Then again, we backtrack to make new decisions. In this case, we need to use the backtracking strategy.

How Does a Backtracking Algorithm Work?

In any backtracking algorithm, the algorithm seeks a path to a feasible solution that includes some intermediate checkpoints. If the checkpoints do not lead to a viable solution, the problem can return to the checkpoints and take another path to find a solution. Consider the following scenario:



1. In this case, S represents the problem's starting point. You start at S and work your way to solution S1 via the midway point M1. However, you discovered that solution S1 is not a viable solution to our problem. As a result, you backtrack (return) from S1, return to M1, return to S, and then look for the feasible solution S2. This process is repeated until you arrive at a workable solution.
2. S1 and S2 are not viable options in this case. According to this example, only S3 is a viable solution. When you look at this example, you can see that we go through all possible combinations until you find a viable solution. As a result, you refer to backtracking as a brute-force algorithmic technique.
3. A "space state tree" is the above tree representation of a problem. It represents all possible states of a given problem (solution or non-solution).

The final algorithm is as follows:

- Step 1: Return success if the current point is a viable solution.
- Step 2: Otherwise, if all paths have been exhausted (i.e., the current point is an endpoint), return failure because there is no feasible solution.
- Step 3: If the current point is not an endpoint, backtrack and explore other points, then repeat the preceding steps.

Applications of Backtracking

- N-queen problem
- Sum of subset problem
- Graph coloring
- Hamilton cycle

N-Queens problem

- The N - Queens problem is to place N - queens in such a manner on an N x N chessboard that no queens attack each other by being in the same row, column, or diagonal.
- Here, we solve the problem for **N = 4 Queens**.

- Before solving the problem, let's know about the movement of the queen in chess.
- In the chess game, a queen can move any number of steps in any direction like vertical, horizontal, and diagonal.
- The only constraint is that it can't change its direction while it's moving.
- In the 4 - Queens problem we have to place 4 queens such as Q1, Q2, Q3, and Q4 on the chessboard, in such a way that no two queens attack each other.
- To solve this problem generally Backtracking algorithm or approach is used.
- In backtracking, start with one possible move out of many available moves, then try to solve the problem.
- If it can solve the problem with the selected move then it will print the solution, else it will backtrack and select some other move and try to solve it.
- If none of the moves works out the claim that there is no solution for the problem.

Algorithm for N-Queens Problem using Backtracking

Step 1 - Place the queen row-wise, starting from the left-most cell.

Step 2 - If all queens are placed then return true and print the solution matrix.

Step 3 - Else try all columns in the current row.

- **Condition 1** - Check if the queen can be placed safely in this column then mark the current cell [Row, Column] in the solution matrix as 1 and try to check the rest of the problem recursively by placing the queen here leads to a solution or not.
- **Condition 2** - If placing the queen [Row, Column] can lead to the solution return true and print the solution for each queen's position.
- **Condition 3** - If placing the queen cannot lead to the solution then unmark this [row, column] in the solution matrix as 0, BACKTRACK, and go back to condition 1 to try other rows.

Step 4 - If all the rows have been tried and nothing worked, return false to trigger backtracking.

The Working of an Algorithm to solve the 4-Queens Problem

- To solve the problem, place a queen in a position and try to find out all the possibilities for any queen being under attack or not.
- Based on the findings place only one queen in each row/column.
- If we found that the queen is under attack at its chosen position, then try for the next position.
- If a queen is under attack at all the positions in a row, we backtrack and change the position of the queen placed prior to the current position.

- Repeat this process of placing a queen and backtracking until all the N queens are placed successfully.

Step 1

As this is the 4-Queens problem, therefore, create a 4×4 chessboard.

Step 2

- Place the Queen Q1 at the left-most position which means row 1 and column 1.
- Mark the cells of the chessboard with cross marks that are under attack from a queen Q1.
- (horizontal, vertical, and diagonal move of the queen)
- The chessboard looks as follows after placing **Q1 at [1, 1] position**:

Q1	×	×	×
×	×		
×		×	
×			×

Step 3

- The possible safe cells for Queen Q2 at row 2 are of column 3 and 4 because these cells do not come under the attack from a queen Q1.
- So, here we place Q2 at the first possible safe cell which is row 2 and column 3.
- Mark the cells of the chessboard with cross marks that are under attack from a queen Q2.
- (horizontal, vertical, and diagonal move of the queen)
- The chessboard looks as follows after placing **Q2 at [2, 3] position**:

Q1	×	×	×
×	×	Q2	×
×	×	×	×
×		×	×

Step 4

- We see that no safe place is remaining for the Queen Q3 if we place Q2 at position [2, 3]. Therefore make position [2, 3] false and backtrack.

Step 5

- So, we place Q2 at the second possible safe cell which is row 2 and column 4.

- Mark the cells of the chessboard with cross marks that are under attack from a queen Q2. (horizontal, vertical, and diagonal move of the queen)
- The chessboard looks as follows after placing **Q2 at [2, 4] position:**

Q1	×	×	×
×	×	×	Q2
×		×	×
×	×		×

Step 6

- The only possible safe cell for Queen Q3 is remaining in row 3 and column 2.
- Therefore, we place Q3 at the only possible safe cell which is row 3 and column 2.
- Mark the cells of the chessboard with cross marks that are under attack from a queen Q3. (horizontal, vertical, and diagonal move of the queen)
- The chessboard looks as follows after placing **Q3 at [3, 2] position:**

Q1	×	×	×
×	×	×	Q2
×	Q3	×	×
×	×	×	×

Step 7

We see that no safe place is remaining for the Queen Q4 if we place Q3 at position [3, 2]. Therefore, make position [3, 2] false and backtrack.

Step 8

- This time we backtrack to the first queen Q1.
- Place the Queen Q1 at column 2 of row 1.
- Mark the cells of the chessboard with cross marks that are under attack from a queen Q1. (horizontal, vertical, and diagonal move of the queen)
- The chessboard looks as follows after placing **Q1 at [1, 2] position:**

×	Q1	×	×
×	×	×	
	×		×
	×		

Step 9

- The only possible safe cell for Queen Q2 is remaining in row 2 and column 4.
- Therefore, we place Q2 at the only possible safe cell which is row 2 and column 4.
- Mark the cells of the chessboard with cross marks that are under attack from a queen Q2. (horizontal, vertical, and diagonal move of the queen)
- The chessboard looks as follows after placing **Q2 at [2, 4] position:**

×	Q1	×	×
×	×	×	Q2
	×	×	×
	×		×

Step 10

- The only possible safe cell for Queen Q3 is remaining in row 3 and column 1.
- Therefore, we place Q3 at the only possible safe cell which is row 3 and column 1.
- Mark the cells of the chessboard with cross marks that are under attack from a queen Q3. (horizontal, vertical, and diagonal move of the queen)
- The chessboard looks as follows after placing **Q3 at [3, 1] position:**

×	Q1	×	×
×	×	×	Q2
Q3	×	×	×
×	×		×

Step 11

- The only possible safe cell for Queen Q4 is remaining in row 4 and column 3.
- Therefore, we place Q4 at the only possible safe cell which is row 4 and column 3.
- The chessboard looks as follows after placing Q3 at [4, 3] position:

×	Q1	×	×
×	×	×	Q2
Q3	×	×	×
×	×	Q4	×

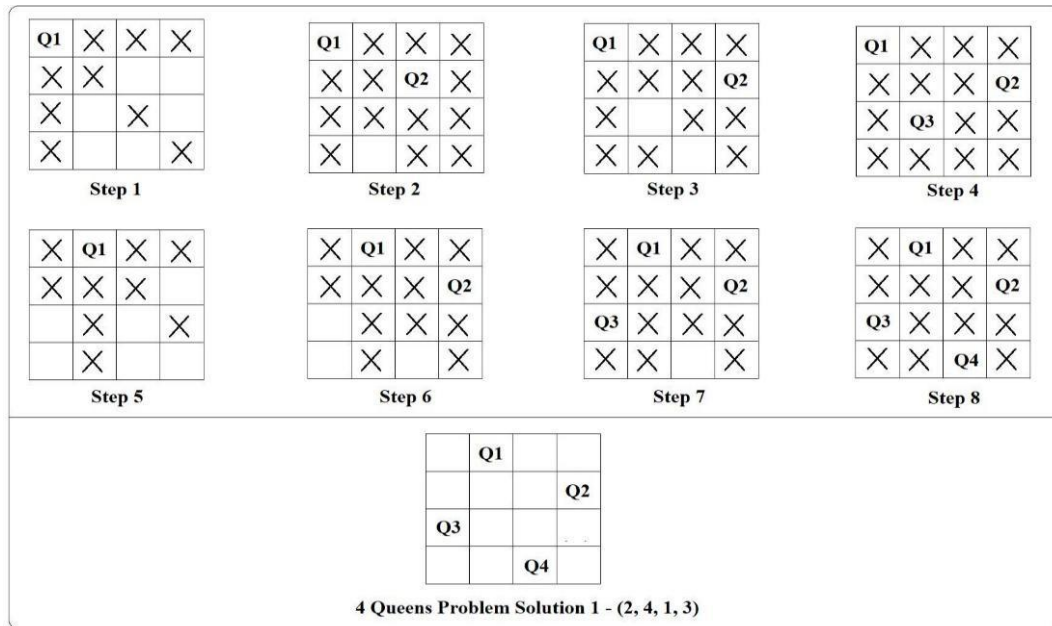
Step 12

- Now, here we got the solution for the 4-queens problem because all 4 queens are placed exactly in each row/column individually.

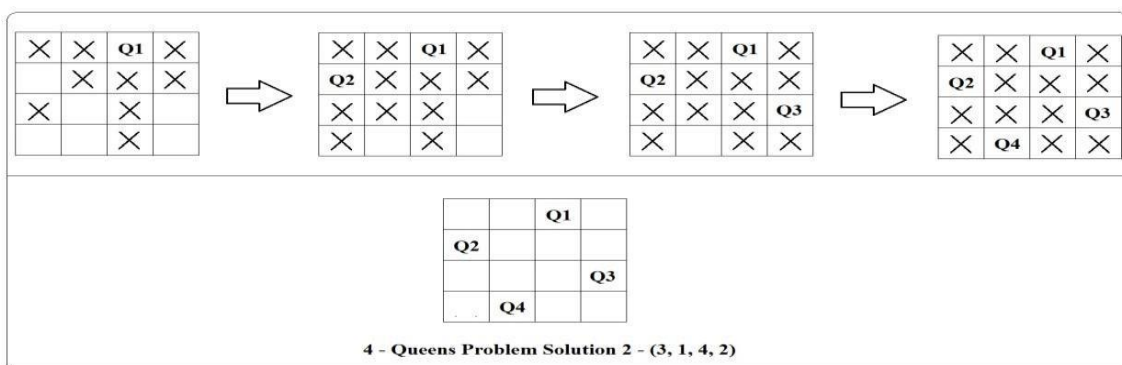
- The N - Queens problem is to place N - queens in such a manner on an N x N chessboard that no queens attack each other by being in the same row, column, or diagonal.

	Q1		
			Q2
Q3			
		Q4	

All these procedures are shown sequentially in the below figure:



- This is not the only possible solution to the problem.
- If we move each queen one step forward in a clockwise manner, we get another solution.
- This is also shown sequentially in the below figure:



- In this example we placed the queens according to rows, we can do the same thing column-wise also.

Space and time complexity: Backtracking algorithm

The time complexity of backtracking depends on the number of times the function calls itself.

For example, if the function calls itself two times, then its time complexity is $O(2^N)$, and if it calls three times, then $O(3^N)$ and so on. Hence the time complexity of backtracking can be defined as $O(K^N)$, where 'K' is the number of times the function calls itself.

N-Queens problem

The function queen iterates n times and for each iteration it calls the function place(row) function the total time complexity would be $O(N^2)$. So we can conclude that the time complexity of N Queens Algorithm is $O(n!)$

Space complexity - $O(N^2)$ Where N is no. of Queens.